FINAL YEAR PROJECT PROPOSAL

WebShield AI

AI-Powered Website Fraud Detection and Trust Analysis System

Submission details	Information
Submitted by	[Student name / team member names]
Registration / roll number	[Registration / roll number(s)]
Degree programme	[Degree programme]
Department	[Department name]
University / institution	[University / institution name]
Supervisor	[Supervisor name]
Academic session	[Academic session]
Submission date	[Submission date]

An intelligent system for fraudulent website detection and explainable risk assessment

1. Title of Project

WebShield AI: AI-Powered Website Fraud Detection and Trust Analysis System

Problem Statement

Users encounter suspicious links through messages, advertisements, online stores and investment promotions. A website can use HTTPS, professional branding or cryptocurrency terminology without establishing that its operator is trustworthy. URL patterns, reported threats and page indicators also have limitations when considered individually. Users need an understandable assessment that shows the available evidence and identifies uncertainty.

WebShield AI will be an AI-powered web application designed to detect fraudulent websites, including phishing pages, deceptive online stores and suspicious investment or cryptocurrency websites. It will combine machine-learning predictions with URL characteristics, domain information, website content and threat-intelligence reports. Users will receive a risk assessment with the evidence behind each finding, helping them make informed decisions before entering credentials or making payments.

2. Objectives

- O1. Develop a web application that accepts website URLs, processes scans asynchronously and stores scan results and supporting evidence.
- O2. Detect indicators of phishing, deceptive storefronts and investment fraud by analyzing URL structure, domain information, page content, threat reports and cryptocurrency-related signals.
- O3. Train and compare Logistic Regression, Random Forest and Histogram Gradient Boosting, evaluate them on unseen labeled data, and integrate the selected model into the detection workflow.
- O4. Present understandable website-risk categories, model estimates, explanations, scan history and downloadable reports through an accessible dashboard.
- O5. Evaluate detection quality, false positives, system reliability and security through reproducible experiments, software testing and documented results.

3. Scope of the Project

The project will cover end-to-end analysis of public HTTP(S) websites: URL validation, DNS and RDAP metadata, bounded website-content collection, threat-intelligence lookup, ML-based classification and explainable risk reporting. Content analysis will examine page text, credential forms, destination domains and suspicious investment claims. Cryptocurrency analysis will identify relevant language and validate candidate wallet or contract addresses. Background processing, scan history, retries, JSON export and provider-status information will complete the user workflow.

Website content will be collected through a separate worker with restricted network access and resource limits. The initial FYP deployment will support a controlled demonstration environment. Wallet transactions, seed-phrase collection, authentication or CAPTCHA bypass, automated takedown and full on-chain tracing are excluded. Screenshot-based brand comparison and a browser extension may be considered as future enhancements.

4. Core Functional Requirements

The following ten core functional requirements describe the intended behavior of the completed project.

FR No.	Functional requirement	Description
FR-01	URL submission and validation	The system shall accept a public HTTP(S) URL and reject malformed inputs, credentials, disallowed ports, internal hostnames and nonpublic or ambiguous IP addresses with a readable error.
FR-02	Asynchronous scan processing	The system shall persist each accepted scan with a unique identifier, process it through a background queue, and expose queued, running, completed, partial or failed status with timestamps.
FR-03	URL and domain analysis	The system shall extract URL features, resolve public domain records and retrieve available RDAP metadata, including domain creation age, while recording missing information explicitly.
FR-04	AI-based fraud detection	The system shall use a trained and evaluated ML model to classify website risk and provide a model estimate using versioned features and thresholds selected on validation data.
FR-05	Threat-intelligence lookup	The system shall check submitted URLs against a configured threat-intelligence provider and record reported threats, source, lookup time, report expiry and provider availability.
FR-06	Website-content analysis	The system shall collect bounded page content through an isolated worker and analyze text, login forms, form-action domains and redirect patterns for potential phishing or deceptive behavior.
FR-07	Cryptocurrency scam indicators	The system shall detect crypto-related content, suspicious investment claims and validated address candidates, and explain how these indicators contribute to the assessment.
FR-08	Risk assessment and explanation	The system shall combine model estimates and supporting evidence into an explained risk category, displaying sources and timestamps and returning inconclusive when evidence is insufficient.
FR-09	History and export	The system shall provide paginated scan history, access to stored reports, scan resubmission and downloadable JSON evidence reports.
FR-10	Safety and configuration	The system shall enforce request limits and safe output handling, keep provider credentials on the backend, and display provider availability and privacy information without exposing secrets.

5. Tools Used in the Project

The project will use the following technologies for application development, model training, inference, testing and deployment.

Category	Technology	Purpose
Languages	TypeScript, JavaScript, Python	Application logic, shared features, ML training and inference.
Frontend	Next.js 16 App Router, React 19, CSS	Responsive dashboard, scan reports, history and settings.
Backend	Node.js, NestJS 11	REST API, scan orchestration and integration with ML inference.
Database	PostgreSQL 16, TypeORM	Persisted scans, JSONB evidence and versioned migrations.
Background queue	Redis 7, BullMQ	Persistent jobs, bounded retries and worker processing.
Machine learning	scikit-learn, pandas, joblib	Three baselines, calibration, evaluation and versioned inference.
Threat intelligence	Google Safe Browsing adapter	Reported-threat lookup through a configured backend integration.
Testing and quality	Vitest, Supertest, Playwright, axe-core, Python unittest, ESLint, Prettier	Unit, API, browser, accessibility and ML pipeline checks.
Deployment and CI	Docker Compose, GitHub Actions	Local services and automated verification workflow.
Development and documentation	npm workspaces, Git, Makefile, Mermaid, Word-compatible documents	Project tasks, version control, diagrams and academic deliverables.

6. Techniques Used in the Project

- Requirement engineering and modular design: align objectives, acceptance criteria, typed API contracts and component boundaries.
- Database design and asynchronous processing: store scan records with indexed history and provenance, and use persistent jobs for background work.
- Feature engineering and data preparation: normalize URLs, derive lexical/domain/content features, validate labels and remove conflicting or duplicate records.
- Leakage-resistant ML evaluation: separate connected domains/campaigns across temporal partitions, calibrate on validation data and reserve the final test set.
- Hybrid detection and secure processing: combine model predictions with domain, content and threat evidence while isolating website-content collection.
- Software verification and accessibility: test functional behavior, failure paths, unsafe inputs, responsive layouts and keyboard or screen-reader access.

7. Methodology Pipeline

Problem identification → literature review → requirements analysis → architecture and database design → dataset acquisition and labeling → preprocessing and feature extraction → application and model development → testing and evaluation → local deployment and documentation.

Phase	Input	Main activity	Output
1. Requirements	Problem and user needs	Review related approaches; define scope, risks and measurable requirements.	Proposal and SRS.
2. Design	Approved requirements	Design API, queue, database, UI and content-worker boundaries.	Architecture, schema and interface designs.
3. Data	Permitted URL datasets and local fixtures	Record source terms and collection dates; review labels; deduplicate and group records.	Dataset card and prepared labeled data.
4. Development	Design and prepared data	Implement domain/content checks, train models and integrate inference, providers and risk reporting.	Integrated detection application and versioned models.
5. Verification	Application and model pipeline	Run unit, integration, browser, accessibility and security checks; verify failure states.	Test evidence and resolved defects.
6. Evaluation	Frozen holdout and predictions	Measure detection metrics, calibration, error patterns and feature-group comparisons.	Reproducible evaluation and model card.
7. Deployment	Verified prototype	Package local services and prepare setup, demonstration and user instructions.	Deployable FYP and final documentation.

Operational Workflow

A user will submit a URL through the Next.js interface. NestJS will validate the input, save the scan in PostgreSQL and add a BullMQ job to Redis. The worker will extract URL features, retrieve domain metadata, query the threat provider and request bounded content analysis from the isolated worker. A versioned ML inference module will produce a prediction. The risk engine will combine the evidence and model estimate into an explained assessment. The frontend will display progress and the saved report; users can revisit history, retry a scan or export its evidence.

Research Workflow

The research dataset will contain URLs, reviewed labels, observation dates, source information and domain/campaign groups. Canonical duplicates and overlapping groups will be controlled before temporal partitioning. Three models will be trained and calibrated, then compared using validation data. The selected model and thresholds will be frozen before final holdout testing and integrated into the scan workflow. Synthetic fixtures will support software testing; detection performance will be measured on independently labeled examples.

8. Requirement-to-Objective Mapping

Objective	Related requirements	Expected result
O1. Safe persistent scan application	FR-01, FR-02, FR-10	Validated submissions, recoverable background processing and protected configuration.
O2. Fraud-indicator detection	FR-03, FR-05, FR-06, FR-07	Combined URL, domain, content, threat-report and crypto evidence.
O3. Evaluated ML classification	FR-04	Three-model comparison, selected model integration and measured holdout results.
O4. Understandable risk reports	FR-08, FR-09	Explained assessments, model estimates, history, resubmission and export.
O5. Verifiable research prototype	FR-04, FR-10; cross-cutting tests	Repeatable experiments, security evidence, model limitations and technical documentation.

9. Evaluation Plan and Expected Deliverables

Evaluation Plan

Functional evaluation will verify URL validation, domain lookup, content extraction, model inference, risk aggregation, lifecycle transitions, history and export. Failure tests will cover provider timeouts, missing metadata, interrupted jobs and partial reports. Security tests will cover unsafe URLs, redirect validation, DNS rebinding defenses, socket destinations, output escaping, resource limits and worker isolation. Browser checks will assess desktop/mobile layouts and keyboard access.

ML evaluation will report confusion matrices, per-class precision, recall and F1, false-positive rate, average precision, ROC-AUC where applicable, Brier score, calibration curves and inference latency. Thresholds will be selected using validation data, with false-positive tradeoffs documented before final testing. Error analysis will consider domain groups, observation age and crypto/non-crypto examples where labels support those comparisons.

The research hypothesis is that combining URL, domain, content and threat-report indicators can improve detection of unseen fraudulent websites over URL-only analysis. Ablation experiments will compare URL-only, URL-plus-domain, additional content and additional threat-intelligence features. Measured results will establish the benefit of each group and the tradeoff between detection and false positives.

Expected Deliverables

- An integrated fraudulent-website detection application with URL/domain analysis, isolated content checks, threat lookup, ML inference and explainable risk reports.
- A documented labeled dataset, reproducible three-model comparison, selected versioned model and measured evaluation results.
- SRS, architecture and database diagrams, dataset/model cards, threat model, test evidence, user manual and final evaluation report.
- Scan history, downloadable JSON reports, provider-status information, demonstration scenarios, screenshots and a repeatable deployment checklist.

Expected Outcome and Project Feasibility

The expected outcome is a working web application that detects indicators of fraudulent websites and gives users an understandable risk assessment before they interact with a suspicious link. The completed workflow will integrate URL and domain analysis, isolated page-content inspection, threat-intelligence lookup, ML inference, cryptocurrency indicators and evidence-based explanations. Reports will support informed decisions about entering credentials, sharing information or making payments.

The project will be developed incrementally through application infrastructure, dataset preparation, baseline comparison, detection-module integration and final verification. Open-source application and ML technologies will support reproducibility and a controlled FYP deployment. The API, queue, content worker and model module will have clear boundaries so each component can be tested and maintained independently.

Research and Operational Considerations

Dataset sources, permissions, label definitions and observation dates will be documented. Domain/campaign separation and temporal holdouts will measure performance on unseen examples. Website content will be collected with bounded resources and restricted network access; provider credentials will remain on the backend. Provider availability and evidence freshness will be visible in reports.

Detection quality will be established through measured evaluation rather than a predetermined accuracy claim. A risk category or model estimate will indicate the evidence available at scan time; an inconclusive result will handle insufficient evidence. This is necessary because website behavior changes and legitimate sites can share characteristics with fraudulent ones.

References

[1] Next.js documentation. Frontend framework and application structure. https://nextjs.org/docs

[2] NestJS documentation. Backend framework and modular application architecture. https://docs.nestjs.com/

[3] scikit-learn: Probability calibration. Methodological basis for probability calibration. https://scikit-learn.org/stable/modules/calibration.html

[4] Google Safe Browsing: Usage restrictions. Conditions to review before enabling the optional provider. https://developers.google.com/safe-browsing/v4/usage-limits